1. Infix to Postfix

Algorithm

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. Else,
…..**3.1** If the precedence of the scanned operator is greater than the precedence of the operator in the stack(or the stack is empty), push it.
…..**3.2** Else, Pop the operator from the stack until the precedence of the scanned operator is less-equal to the precedence of the operator residing on the top of the stack. Push the scanned operator to the stack.
4. If the scanned character is an '(', push it to the stack.
5. If the scanned character is an ')', pop and output from the stack until an '(' is encountered.
6. Repeat steps 2-6 until infix expression is scanned.
7. Pop and output from the stack until it is not empty.

2. Postfix to Infix

```
Input : abc++

Output : (a + (b + c))


Input  : ab*c+

Output : ((a*b)+c)
```

1.While there are input symbol left
…1.1 Read the next symbol from the input.
2.If the symbol is an operand
…2.1 Push it onto the stack.
3.Otherwise,
…3.1 the symbol is an operator.
…3.2 Pop the top 2 values from the stack.
…3.3 Put the operator, with the values as arguments and form a string.
…3.4 Push the resulted string back to stack.
4.If there is only one value in the stack
…4.1 That value in the stack is the desired infix string.

3. Solving postfix expression

The Postfix notation is used to represent algebraic expressions. The expressions written in postfix form are evaluated faster compared to infix notation as parenthesis are not required in postfix.
Following is algorithm for evaluation postfix expressions.
1) Create a stack to store operands (or values).
2) Scan the given expression and do following for every scanned element.
…..a) If the element is a number, push it into the stack
…..b) If the element is a operator, pop operands for the operator from stack. Evaluate the operator and push the result back to the stack
3) When the expression is ended, the number in the stack is the final answer

"2 3 1 * + 9 -"

```
abc-+de-fg-h+/*
```

Expression	Stack
abc-+de-fg-h+/*	NuLL
bc-+de-fg-h+/*	"a"
c-+de-fg-h+/*	"b" "a"
-+de-fg-h+/*	"c" "b" "a"
+de-fg-h+/*	"b - c" "a"
de-fg-h+/*	"a+b-c"
e-fg-h+/*	"d" "a+b-c"
-fg-h+/*	"e" "d" "a+b-c"
fg-h+/*	"d - e" "a+b-c"
g-h+/*	"f" "d - e" "a+b-c"
-h+/*	"g" "f" "d - e" "a+b-c"
h+/*	"f-g" "d - e" "a+b-c"
+/*	"h" "f-g" "d - e" "a+b-c"

/*	"f-g+h" "d - e" "a+b-c"
*	"(d-e)/(f-g-h)" "a+b-c"
Null	"(a+b-c)*(d-e)/(f-g+h)"

Ans = (a+b-c)*(d-e)/(f-g+h)

```
for (i = 0; exp[i]; ++i)

 {
     // If the scanned character is an operand (number here),
     // push it to the stack.
     if (isdigit(exp[i]))
         push(stack, exp[i] - '0');

     // If the scanned character is an operator, pop two
     // elements from stack apply the operator
     else
     {
         int val1 = pop(stack);
         int val2 = pop(stack);
         switch (exp[i])
         {
         case '+': push(stack, val2 + val1); break;
         case '-': push(stack, val2 - val1); break;
         case '*': push(stack, val2 * val1); break;
         case '/': push(stack, val2/val1); break;
         }
     }
 }
```

4. Infix to Prefix

```
Input : A * B + C / D
Output : + * A B/ C D


Input : (A - B/C) * (A/K-L)
Output : *-A/BC-/AKL
```

Step 1: Reverse the infix expression i.e A+B*C will become C*B+A. Note while reversing each '(' will become ')' and each ')' becomes '('.
Step 2: Obtain the postfix expression of the modified expression i.e CB*A+.
Step 3: Reverse the postfix expression. Hence in our example prefix is +A*BC.

5. Prefix to Infix

Algorithm for Prefix to Infix:

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
  Create a string by concatenating the two operands and the operator between them.
  string = (operand1 + operator + operand2)
  And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

```
▪ Input :  Prefix :  *+AB-CD
▪ Output : Infix : ((A+B)*(C-D))
```

6. Prefix to Postfix

```
Input :  Prefix :  *+AB-CD
Output : Postfix : AB+CD-*
Explanation : Prefix to Infix :  (A+B) * (C-D)
              Infix to Postfix :  AB+CD-*

Input :  Prefix :  *-A/BC-/AKL
Output : Postfix : ABC/-AK/L-*
Explanation : Prefix to Infix :  A-(B/C)*(A/K)-L
              Infix to Postfix : ABC/-AK/L-*
```

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
  Create a string by concatenating the two operands and the operator after them.
  string = operand1 + operand2 + operator
  And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

7. Postfix to Prefix

```
Input :  Postfix : AB+CD-*
Output : Prefix :  *+AB-CD
Explanation : Postfix to Infix : (A+B) * (C-D)
              Infix to Prefix :  *+AB-CD

Input :  Postfix : ABC/-AK/L-*
Output : Prefix :  *-A/BC-/AKL
Explanation : Postfix to Infix : ((A-(B/C))*((A/K)-L))
              Infix to Prefix :  *-A/BC-/AKL
```

- *Read the Postfix expression from left to right*
- *If the symbol is an operand, then push it onto the Stack*
- *If the symbol is an operator, then pop two operands from the Stack*
  Create a string by concatenating the two operands and the operator before them.
  string = operator + operand2 + operand1
  And push the resultant string back to Stack

- *Repeat the above steps until end of Prefix expression.*

Conversion of Infix to Prefix using Stack

K + L - M * N + (O^P) * W/U/V * T + Q

If we are converting the expression from infix to prefix, we need first to reverse the expression.

The Reverse expression would be:

Q + T * V/U/W *) P^O(+ N*M - L + K

To obtain the prefix expression, we have created a table that consists of three columns, i.e., input expression, stack, and prefix expression. When we encounter any symbol, we simply add it into the prefix expression. If we encounter the operator, we will push it into the stack.

Input expression	Stack	Prefix expression
Q		Q
+	+	Q
T	+	QT
*	+*	QT
V	+*	QTV
/	+*/	QTV
U	+*/	QTVU
/	+*//	QTVU
W	+*//	QTVUW
*	+*//*	QTVUW
)	+*//*)	QTVUW
P	+*//*)	QTVUWP
^	+*//*)^	QTVUWP
O	+*//*)^	QTVUWPO
(	+*//*	QTVUWPO^
+	++	QTVUWPO^*//*

N	++	QTVUWPO^*//*N
*	++*	QTVUWPO^*//*N
M	++*	QTVUWPO^*//*NM
-	++-	QTVUWPO^*//*NM*
L	++-	QTVUWPO^*//*NM*L
+	++-+	QTVUWPO^*//*NM*L
K	++-+	QTVUWPO^*//*NM*LK
		QTVUWPO^*//*NM*LK+-++

The above expression, i.e., QTVUWPO^*//*NM*LK+-++, is not a final expression. We need to reverse this expression to obtain the prefix expression.

Rules for the conversion of infix to prefix expression:

- First, reverse the infix expression given in the problem.
- Scan the expression from left to right.
- Whenever the operands arrive, print them.
- If the operator arrives and the stack is found to be empty, then simply push the operator into the stack.
- If the incoming operator has higher precedence than the TOP of the stack, push the incoming operator into the stack.
- If the incoming operator has the same precedence with a TOP of the stack, push the incoming operator into the stack.
- If the incoming operator has lower precedence than the TOP of the stack, pop, and print the top of the stack. Test the incoming operator against the top of the stack again and pop the operator from the stack till it finds the operator of a lower precedence or same precedence.
- If the incoming operator has the same precedence with the top of the stack and the incoming operator is ^, then pop the top of the stack till the condition is true. If the condition is not true, push the ^ operator.
- When we reach the end of the expression, pop, and print all the operators from the top of the stack.
- If the operator is ')', then push it into the stack.
- If the operator is '(', then pop all the operators from the stack till it finds) opening bracket in the stack.
- If the top of the stack is ')', push the operator on the stack.
- At the end, reverse the output.